

RETO PRÁCTICO – REACT

Carrito de Compras con Validaciones de Stock

 Duración Según cronograma del instructor	 Tecnología React (recomendada)	 Modalidad Individual	 Fecha _____
--	--	--	---

Nombre del Aprendiz: _____	Ficha: _____ Instructor: Daniel Alfonso Martínez Payán
Enlace del repositorio público (GitHub/GitLab): _____	Tecnología usada: ☐ React ☐ HTML + CSS + JS

PROBLEMÁTICA

La tienda virtual **TIENDA PALMIRA** vende productos típicos de la región. Su carrito de compras actual presenta fallas graves: los clientes pueden pedir más unidades de las que hay en bodega, escribir cantidades negativas, en cero o con letras (como la “e” de la notación científica), y los totales no se actualizan correctamente cuando un mismo producto se agrega varias veces. Esto genera pedidos imposibles de despachar, reclamos y pérdida de confianza.

Como aprendiz de desarrollo, usted debe construir un **carrito de compras** que valide toda la información en el **frontend** y avise al usuario con **toasts** (mensajes emergentes) cada vez que intente hacer algo no permitido.

LO QUE SE REQUIERE

Construya el carrito de compras cumpliendo al pie de la letra cada punto de este documento. Puede apoyarse en **cualquier medio** (documentación, tutoriales, videos, compañeros, herramientas de inteligencia artificial): lo que se evalúa es que la aplicación funcione y cumpla las validaciones. Entre todos los trabajos se elegirá **el mejor carrito**.

Tecnología: se espera que lo desarrolle en **React**. También se acepta **HTML, CSS y JavaScript** puros, pero esto **resta puntos** en el criterio 7 de la rúbrica.

Entrega: el código debe estar en un **repositorio público** (GitHub o GitLab) para que el instructor o cualquier otro aprendiz lo pueda clonar y ejecutar. Escriba el enlace en el encabezado de este documento y en el README del proyecto.

PUNTO 1 – Datos del catálogo: array JSON en el frontend

El catálogo debe venir de un **array de objetos JSON** escrito en el frontend (sin base de datos ni API). Use exactamente estos productos para que el instructor pueda comparar resultados:

id	nombre	precio (COP)	stock
1	Café de Huila 500 g	\$28.500	8
2	Panela orgánica 1 kg	\$9.800	3
3	Arepa de choclo x6	\$12.000	12
4	Chocolate de mesa	\$15.400	1
5	Bocadillo veleño	\$6.500	20
6	Aguardiente 750 ml	\$62.000	5

```
const PRODUCTOS = [
  { id: 1, nombre: "Café de Huila 500 g", precio: 28500, stock: 8 },
  { id: 2, nombre: "Panela orgánica 1 kg", precio: 9800, stock: 3 },
  { id: 3, nombre: "Arepas de choclo x6", precio: 12000, stock: 12 },
  { id: 4, nombre: "Chocolate de mesa", precio: 15400, stock: 1 },
  { id: 5, nombre: "Bocadillo veleño", precio: 6500, stock: 20 },
  { id: 6, nombre: "Aguardiente 750 ml", precio: 62000, stock: 5 },
];
```

PUNTO 2 – Navbar e ícono del carrito (10 pts)

- Cree una barra de navegación (navbar) fija en la parte superior con el nombre de la tienda a la izquierda.
- El **ícono del carrito** debe estar ubicado a la **derecha** del navbar.
- Sobre el ícono muestre un **contador** con el total de unidades que hay en el carrito (se oculta si es 0).
- Al hacer clic en el ícono se abre el carrito (panel lateral, modal o página) y debe poder cerrarse.

PUNTO 3 – Catálogo y botón Agregar (10 pts)

- Muestre cada producto con su nombre, precio y stock disponible, un **campo numérico de cantidad** (inicia en 1) y el botón **Agregar**.
- Si el producto ya está en el carrito, al agregarlo de nuevo **no se crea otra línea**: se suma la cantidad a la existente.
- Si el producto se queda sin unidades disponibles, el botón Agregar se deshabilita.

PUNTO 4 – Validaciones del campo numérico (20 pts)

Todo campo de cantidad (en el catálogo y en el carrito) debe cumplir:

- **No permitir letras**, en especial la **"e"** / **"E"** (el número de Euler y la notación científica que el input numérico deja escribir).
- No permitir los signos **+** y **-**, ni punto ni coma: solo **números enteros positivos**.
- No permitir **0** ni **números negativos**.
- Si el usuario **pega** un texto, solo se acepta si son únicamente dígitos.
- Evitar que la rueda del mouse cambie el valor sin querer.

Pista: investigue los eventos `onKeyDown`, `onChange` y `onPaste`, y cómo se bloquea una tecla con `preventDefault`.

PUNTO 5 – Stock máximo con toast (15 pts)

- La cantidad de un producto en el carrito **nunca puede superar su stock**.
- Esto aplica en tres lugares: al escribir un número mayor en el campo, al pulsar el botón **+** y al **agregar** de nuevo un producto que ya está en el carrito.
- Cuando el usuario intente superar el stock, se corrige al máximo y aparece un **toast** con el mensaje: "Este es el máximo de producto disponible en stock".

PUNTO 6 – Mínimo 1 y eliminación con toast (15 pts)

- Dentro del carrito, cada producto tiene botones **-** y **+** y un campo de cantidad.
- Si la cantidad es **1** y el usuario pulsa **-** (o escribe 0), no baja: aparece un **toast** que dice que esta es la cantidad mínima y pregunta si desea **eliminar el producto**, con un botón para confirmar.
- Al confirmar, el producto se quita del carrito y el total se recalcula. Debe existir también un botón para quitar el producto directamente.

Los toasts deben desaparecer solos después de unos segundos y poder cerrarse manualmente. No se aceptan `alert()` del navegador.

PUNTO 7 – Subtotales y total de la compra (15 pts)

- Cada línea del carrito muestra su **subtotal** (precio × cantidad).
- Al final se muestra el **total de la compra** (suma de los subtotales) y el total de unidades.

- Los valores se actualizan al instante al agregar más del mismo producto, cambiar cantidades o eliminar, y se muestran con formato de moneda (COP).

PUNTO 8 – Repositorio y evidencias (5 pts + criterio 7)

- Suba el proyecto a un **repositorio público** y verifique que se pueda clonar (git clone) en otro computador.
- Incluya un **README.md** con: nombre del aprendiz y ficha, tecnología usada, pasos para instalar y ejecutar, y las **capturas de pantalla** de las funcionalidades.
- Diligencie la tabla de evidencias de este documento con el nombre de la captura o la ruta dentro del repositorio.

Nombre sugerido del repositorio: **Apellido_Nombre_CarritoReact**

CASOS DE PRUEBA DEL INSTRUCTOR

El instructor o aprendiz clonará el repositorio y ejecutará estas pruebas. Úselas para autoevaluarse antes de entregar:

#	Acción	Resultado esperado
1	Teclear e, E, +, -, . o , en cualquier campo de cantidad.	La tecla no escribe nada en el campo.
2	Pegar -5, 3e2 o abc en un campo de cantidad.	No se pega (solo se aceptan dígitos).
3	Escribir 0 en el campo de cantidad de la tarjeta del producto.	No lo acepta, conserva el valor anterior y muestra un toast de cantidad mínima 1.
4	Escribir 0 en el campo de cantidad dentro del carrito.	La cantidad no cambia y aparece el toast del mínimo con opción de eliminar.
5	Escribir 999 en un producto con stock 8.	Se corrige a 8 y aparece el toast: “Este es el máximo de producto disponible en stock”.
6	Panela (stock 3): agregar 2 unidades y luego agregar 2 más.	Queda en 3 (no en 4) y aparece el toast de máximo disponible.
7	En el carrito, pulsar + cuando la cantidad ya es igual al stock.	No sube y aparece el toast de máximo disponible.
8	En el carrito, pulsar – cuando la cantidad es 1.	Toast: es la cantidad mínima, ¿desea eliminar el producto? Al confirmar, el producto sale y el total se recalcula.
9	Agregar dos veces el mismo producto.	Una sola línea en el carrito con las cantidades sumadas (sin duplicados).
10	Agregar Café x2, Panela x3 y Arepa x1.	Subtotales: \$57.000, \$29.400 y \$12.000. Total: \$98.400.
11	Agregar todo el stock de un producto (ej. Chocolate x1).	El botón Agregar de ese producto queda deshabilitado (sin stock disponible).
12	Agregar productos y observar la barra de navegación.	El ícono del carrito está a la derecha y su contador muestra la suma de unidades.

EVIDENCIAS DEL APRENDIZ

Adjunte una captura por funcionalidad (en el README o en una carpeta /evidencias del repositorio):

#	Funcionalidad	Captura (nombre o ruta)	¿Funciona? (Sí/No)
1	Navbar e ícono con contador		
2	Agregar producto desde el catálogo		
3	Bloqueo de la tecla "e" y de negativos / 0		
4	Toast de stock máximo		
5	Toast de cantidad mínima con opción de eliminar		
6	Subtotales y total con varios productos		
7	Producto eliminado y total recalculado		

Enlace del repositorio público: _____

Enlace de despliegue (opcional): _____

VERIFICACIÓN Y ENTREGA

✓	Criterio de verificación	¿Cumple?
☐	La aplicación corre con los comandos npm install y npm run dev (o abre con Live Server si es HTML puro).	
☐	El repositorio es público y se puede clonar sin pedir credenciales.	
☐	El catálogo sale de un array JSON con id, nombre, precio y stock.	
☐	El ícono del carrito está en la navbar, a la derecha, con contador de unidades.	
☐	Ningún campo de cantidad acepta letras (especialmente la e), signos, puntos, comas, 0 ni negativos.	
☐	Nunca se supera el stock y siempre aparece el toast de máximo disponible.	
☐	Con cantidad 1, al restar (o escribir 0) aparece el toast del mínimo con opción de eliminar.	
☐	Los subtotales y el total cuadran al agregar más del mismo producto, cambiar cantidades o eliminar.	
☐	El README incluye el enlace del repositorio, las instrucciones y las capturas de evidencia.	

RÚBRICA DE EVALUACIÓN

100 puntos = 5.0 | Nota mínima aprobatoria: 3.0 / 5.0

#	Criterio	Pts	Excelente (4.5–5.0)	Aceptable (3.0–4.4)	Insuficiente (<3.0)	Nota
1	Navbar e ícono del carrito a la derecha con contador de unidades	10	Navbar completo, ícono a la derecha, contador correcto y panel del carrito que abre y cierra.	Ícono presente, pero sin contador o con ubicación/funcionamiento incompleto.	No hay navbar o el ícono no abre el carrito.	
2	Catálogo desde array JSON y función de agregar (sin duplicar líneas)	10	Productos renderizados desde el array; agregar suma cantidades en una sola línea por producto.	Funciona, pero duplica líneas o tiene errores con algún producto.	Datos quemados en el HTML o agregar no funciona.	
3	Campo numérico: bloqueo de e, +, -, ., ,, pegado inválido, 0 y negativos	20	Se bloquean todos los casos de teclas y valores inválidos (casos de prueba 1 al 4) sin romper la experiencia.	Bloquea la mayoría; se le cuela la "e", el 0 o un negativo en algún campo.	Acepta letras, negativos o 0 de forma generalizada.	
4	Validación de stock máximo con toast	15	Nunca supera el stock (campo, botón + y agregar repetido) y siempre muestra el toast de máximo.	Limita el stock, pero falta el toast o falla en alguna de las tres vías.	Permite superar el stock.	
5	Mínimo 1 y eliminación con toast de confirmación	15	Con cantidad 1, - y 0 muestran el toast del mínimo con acción de eliminar; al confirmar, se elimina y se recalcula.	Muestra aviso, pero no con toast, o elimina sin confirmar.	Permite 0 o negativos, o elimina sin avisar.	
6	Subtotales y total de la compra	15	Subtotal por línea y total correctos en todos los casos (incluye agregar más del mismo producto y eliminar).	Cálculos correctos en la mayoría; falla en algún caso de actualización.	Totales incorrectos o ausentes.	
7	Tecnología y calidad del código	10	Hecho en React : componentes, hooks/estados bien usados, código ordenado y legible.	React con código desordenado o sin separar componentes.	HTML, CSS y JavaScript puro (no cumple la tecnología recomendada) o código ilegible.	
8	Repositorio público y evidencias	5	Repositorio público que clona y corre, README con instrucciones y capturas de cada funcionalidad.	Repositorio público, pero sin README o con pocas evidencias.	Repositorio privado, enlace roto o sin evidencias.	
NOTA FINAL		100	Nota = puntos obtenidos ÷ 20 (100 puntos = 5.0) · Nota mínima aprobatoria: 3.0 / 5.0			

SELECCIÓN DEL MEJOR CARRITO

Los trabajos con mayor puntaje compiten por el reconocimiento de **mejor carrito de compras**. En caso de empate, el instructor desempata con estos aspectos (no suman a la nota):

- Experiencia de usuario: diseño claro, responsive (se ve bien en celular) y mensajes de toast bien redactados.
- Calidad del código: componentes reutilizables (por ejemplo, un componente de cantidad y otro de toast), nombres claros y sin código repetido.
- Accesibilidad básica: foco visible, etiquetas en los botones y toasts anunciados por lectores de pantalla.
- Valor agregado: persistencia del carrito (localStorage), animaciones sutiles, pruebas o despliegue en línea.

Firma del Aprendiz

Documento: _____

Instructor Evaluador

Daniel Alfonso Martínez Payán